



Programming Challenge

Product Engineer (Operaciones / WMS)

Introducción

Estamos construyendo un módulo para un **WMS** (Warehouse Management System): el software que gobierna la operación de un depósito. En un depósito, el stock no vive "en un solo lugar": está repartido en **ubicaciones**, y cada ubicación cumple un rol distinto.

Hay dos tipos de ubicaciones que nos importan acá:

- **Ubicaciones de *picking***: chicas y de fácil acceso, donde el operario va a buscar producto para armar los pedidos.
- **Ubicaciones de *reserva***: donde se guarda el grueso del stock (racks altos, pallets, etc.).

Un mismo producto (SKU) puede estar en varias ubicaciones a la vez. Cada SKU que se maneja en una ubicación de picking tiene un **mínimo** y un **máximo** definidos. La operación es simple de describir pero llena de detalles: cuando el stock disponible en una ubicación de picking cae **por debajo del mínimo**, hay que **reabastecerla** — mover producto desde una o más ubicaciones de reserva hacia la de picking, apuntando a llevarla hasta su máximo.

Tu trabajo es construir el módulo que **modela el inventario** y **gestiona el reabasto**. Apuntá a una solución pensada para la realidad de un depósito.

Stack

Ya te damos un [template de microservicio](#) (Spring Boot 3.5.4 / Java 24 / Gradle multi-módulo con arquitectura hexagonal).

El template trae una **feature de ejemplo (User) completa y funcionando** — modelo de dominio, puertos, servicio, controller REST, DTOs, documentación OpenAPI y un test de integración. **Usala como mapa:** te muestra todas las convenciones del proyecto (cómo se separan los módulos `domain` / `api` / `infra`, cómo se cablean los beans, cómo se expone un endpoint, cómo se documenta con OpenAPI).

- Levantar: `./run.sh` (o `./gradlew bootRun`).
- API base: <http://localhost:8080/api/templates>
- Swagger UI: <http://localhost:8080/api/templates/documentation>

La persistencia puede ser in-memory como en el ejemplo `User`; no necesitás base de datos para el core.

Si tenés cualquier problema para compilar o acceder a dependencias, avisanos — no queremos que pierdas tiempo con el setup.

Objetivo

La forma es abierta; el comportamiento es concreto. Buscamos una solución que logre un buen balance entre:

1. Facilidad para **agregar** features nuevas.
2. Facilidad para **mantener** las existentes.
3. **Velocidad** de entrega (time to market).
4. Facilidad para **testear** en aislamiento (manual y automático).
5. Corrección de las **reglas de negocio**.
6. Claridad de la **API** (que otro dev la entienda sin leer el código).

No hay una única respuesta correcta. Vas a notar que hay decisiones que el enunciado **no** define: tomalas vos y seguí. Eso también es parte de lo que evaluamos.

Criterios de aceptación

1. **Todos los endpoints funcionan de punta a punta** y son probables vía Swagger o `curl`.
2. **Cada error devuelve el status HTTP correcto** (400 / 404 / 409 / ...) con un mensaje claro. No hay 500 genéricos para casos esperables.
3. **La API está documentada** en OpenAPI/Swagger.
4. **Las reglas de negocio del reabasto están cubiertas por tests**.

Modelo de dominio

Modelalo como te parezca (esto es solo el "qué", no el "cómo"). Lo que sigue es el **mínimo indispensable** para que el reabasto funcione — **no es una lista cerrada**. Si ves que el dominio pide más para resolverlo bien, sumá lo que consideres.

Como mínimo vas a necesitar estos conceptos:

- **Location** — una ubicación del depósito.
 - **code** (identificador único, ej: **PICK-01**)
 - **type** — enum: **PICKING** | **RESERVE**.
- **InventoryItem** — cuánto hay de un SKU en una ubicación (el concepto de *quant* en un WMS).
 - **sku**, la ubicación a la que pertenece, y **quantity**
- **ReplenishmentRule** — los umbrales de reabasto de un SKU en una ubicación de picking.
 - **sku**, **locationCode** (debe ser de tipo **PICKING**), **min**, **max**
- **ReplenishmentTask** — una tarea de reabasto generada por el sistema.
 - **id**, **sku**, **fromLocation**, **toLocation**, **quantity**, **status**
 - **Máquina de estados (FSM)**: **OPEN** → **CONFIRMED** y **OPEN** → **CANCELLED**. Los estados **CONFIRMED** y **CANCELLED** son terminales (read-only).

Datos iniciales (seed)

Dejá un **seeder** que precargue el siguiente escenario al arrancar la app. Documentá en el README cómo correrlo.

Ubicaciones: **PICK-01**, **PICK-02** (picking) · **RSV-01**, **RSV-02**, **RSV-03** (reserva)

Reglas de reabasto:

SKU	Ubicacion	Min	Max
SKU-100	PICK-01	20	100
SKU-200	PICK-01	10	50
SKU-300	PICK-02	30	120

Stock inicial:

SKU	Ubicación	Cantidad
SKU-100	PICK-01	5
SKU-200	PICK-01	40
SKU-300	PICK-02	10
SKU-100	RSV-01	60
SKU-100	RSV-02	50
SKU-300	RSV-03	70

Endpoints

URLs relativas a la base [/api/templates](#). Definí los DTOs de request/response como te parezca (los ejemplos son sugerencias); lo importante es el comportamiento.

1. Crear ubicación

POST [/locations](#)

Crea una ubicación de depósito.

```
{ "code": "PICK-01", "type": "PICKING" }
```

Reglas:

1. `code` es único; si ya existe → error.
2. `type` debe ser `PICKING` o `RESERVE`.

2. Listar ubicaciones

GET [/locations](#)

Devuelve todas las ubicaciones.

3. Cargar stock

POST /stock

Establece la cantidad disponible de un SKU en una ubicación.

```
{ "sku": "SKU-100", "locationCode": "PICK-01", "quantity": 5 }
```

Reglas:

1. La ubicación debe existir → si no, 404.
2. `quantity` no puede ser negativa.

4. Consultar stock

GET /stock?sku={sku}

Devuelve el stock de un SKU en todas sus ubicaciones. (Podés soportar también `?location={code}.`)

5. Definir regla de reabasto

POST /replenishment-rules

Define el min/máx de un SKU en una ubicación de picking.

```
{ "sku": "SKU-100", "locationCode": "PICK-01", "min": 20, "max": 100 }
```

Reglas:

1. La ubicación debe existir y ser de tipo `PICKING` → si no, error.
2. $0 \leq \text{min} \leq \text{max}$.
3. No puede haber dos reglas para el mismo SKU + ubicación.

6. Mover stock

POST /stock/move

Mueve una cantidad de un SKU de una ubicación a otra. **Es la operación base de consistencia del sistema.**

```
{ "sku": "SKU-100", "from": "RSV-01", "to": "PICK-01", "quantity": 20 }
```

Reglas:

1. Ambas ubicaciones deben existir.

2. Debe haber stock suficiente en el origen → si no, 409/400 con mensaje claro.
3. La operación es **atómica**: o se descuenta del origen y se suma al destino, o no pasa nada. Nunca a medias, nunca negativo.

7. Evaluar y generar tarea de reabasto

POST /replenishment/tasks

Dado un SKU en una ubicación de picking, evalúa si necesita reabasto y, de ser así, genera una **tarea de reabasto** que lo lleve de vuelta a su máximo trayendo stock desde reserva. Resuelve los casos que la operación real implica.

```
{ "sku": "SKU-100", "locationCode": "PICK-01" }
```

8. Listar tareas de reabasto

GET /replenishment/tasks

Lista las tareas.

9. Confirmar tarea de reabasto

POST /replenishment/tasks/{id}/confirm

Confirma una tarea de reabasto: **mueve el stock** de reserva a picking (reutiliza la operación del endpoint #6) y cierra la tarea.

10. Cancelar tarea de reabasto

POST /replenishment/tasks/{id}/cancel

Cancela una tarea de reabasto pendiente. No mueve stock.

Nice to have (opcional)

No hace falta que hagas nada de esto. Si te sobra tiempo, elegí **una** cosa y hacela bien:

- POST /replenishment/scan — escanea **todo** el depósito y genera todas las tareas de reabasto pendientes de una sola pasada.

- Registrar los movimientos de stock como un **historial trazable** (qué se movió, cuándo, de dónde a dónde).
- **Persistencia real** con una base de datos relacional (**PostgreSQL** o similar) e integridad referencial, en lugar de in-memory.
- Lo que se te ocurra que le sume a la operación. Sorprendenos.

Uso de IA

Está permitido y lo alentamos — es parte de cómo trabajamos.

Entregables

1. **El repo** con tu solución (que compile y corra).
2. Un **README** con cómo levantar la app, cargar el seed y probar el flujo.

Expectativas

No esperamos perfección: preferimos pocas cosas bien resueltas y prolijas antes que muchas a medias. Si tenés que elegir entre terminar algo bien o empezar tres cosas, elegí lo primero.

¡Éxitos! 🚀

Link al repositorio / Template

<https://github.com/Franco1312/wms-challenge>